Docker Mastery: From Zero to Hero

Part 1: The Fundamentals (Beginner)

Goal: Understand the "why" behind Docker and run your first container.

Module 1: Introduction to Containerization

- **The Problem:** "It works on my machine!" - The challenge of environment inconsistency.
- **The Solution:** What is Containerization? (Vs. Physical Servers and Virtual Machines)
- **Docker vs. VMs:** Lightweight, speed, and resource efficiency.
- **Key Docker Concepts:** Images, Containers, Registries (Docker Hub).
- **The Docker Ecosystem:** Docker Desktop, Docker Engine, Docker CLI.

Module 2: Getting Started & Installation

- Installing Docker Desktop (Windows/Mac) or Docker Engine (Linux).
- Verifying the installation: `docker --version`
- Running the obligatory `docker run hello-world`
- Introduction to the Docker CLI.

Module 3: The Dockerfile - Building Custom Images

- What is a Dockerfile? The blueprint for your image.
- Common Instructions:
  - `FROM` (The base image)
  - `RUN` (Executing commands during build)
  - `COPY` & `ADD` (Adding files from your host)
  - `WORKDIR` (Setting the working directory)
  - `EXPOSE` (Documenting the port)
  - `CMD` & `ENTRYPOINT` (Defining the default command to run)
- **Hands-on Lab 1:** Create a Dockerfile for a simple Python/Node.js web application.

Module 4: Container Management Basics

- Core Commands:
  - `docker run` (flags: `-d`, `-p`, `-v`, `--name`, `-it`)
  - `docker ps` (`docker ps -a`)
  - `docker stop` / `docker start` / `docker restart`
  - `docker rm` (removing containers)
  - `docker logs`
  - `docker exec` (running a command in a running container)

- **Hands-on Lab 2:** Run the custom image from Lab 1, map a port, run it in detached mode, and view its logs.

Part 2: Core Concepts & Development Workflow (Intermediate)

Goal: Master the Docker development workflow and manage data effectively.

Module 5: Docker Images Deep Dive

- Image Layers and the Union File System.
- Efficient Dockerfiles: Leveraging layer caching, using `.dockerignore`.
- Multi-stage builds (for reducing final image size, e.g., compiling code in one stage and running it in a smaller one).
- Image tagging best practices (`my-app:1.0`, `my-app:latest`).
- Commands: `docker build`, `docker images`, `docker rmi`, `docker tag`.

Module 6: Data Persistence: Volumes & Bind Mounts

- The problem: Containers are ephemeral. Data is lost when they stop.
- **Bind Mounts:** Mapping a host directory to a container directory. Perfect for development (live code reload).
- **Volumes:** Docker-managed storage, decoupled from the host filesystem. Perfect for production.
- Commands: `docker volume create`, `docker volume ls`, `docker volume inspect`.
- **Hands-on Lab 3:** Use a bind mount for development with live reload. Then, create a volume for a database (e.g., PostgreSQL).

Module 7: Networking in Docker

- How containers talk to each other and the outside world.
- Default Networks: `bridge`, `host`, `none`.
- Creating a custom bridge network for isolated application stacks.
- Connecting containers by name (Docker's embedded DNS).
- Commands: `docker network create`, `docker network ls`, `docker network connect`.
- **Hands-on Lab 4:** Create a multi-container application (e.g., a web app and a separate database) on a custom network and make them communicate.

Module 8: Docker Compose: The Multi-Container Orchestrator

- What is Docker Compose? Defining and running multi-container applications with a YAML file.
- The `docker-compose.yml` structure: `services`, `networks`, `volumes`.

- Key service directives: `image`, `build`, `ports`, `volumes`, `environment`, `depends_on`.
- Essential commands: `docker-compose up`, `docker-compose down`, `docker-compose ps`, `docker-compose logs`.
- **Hands-on Lab 5:** Convert the multi-container app from Lab 4 into a `docker-compose.yml` file.

Part 3: Advanced Topics & Production Ready (Advanced)

Goal: Prepare for deploying and managing Docker in production environments.

Module 9: Docker Hub & Private Registries

- Pushing images to Docker Hub.
- Introduction to private registries (Docker Trusted Registry, Azure Container Registry, AWS ECR, Google Container Registry).
- Logging into a registry from the CLI (`docker login`).
- Best practices for image tagging and versioning in a team.

Module 10: Security Best Practices

- Running containers as a non-root user.
- Scanning images for vulnerabilities (using `docker scan`).
- Keeping your base images updated.
- Minimizing attack surface (using minimal base images like Alpine Linux).
- The principle of least privilege.

Module 11: Introduction to Container Orchestration

- **The Problem:** Managing 10s, 100s, or 1000s of containers across multiple hosts.
- **The Solution:** Orchestrators.
- **Docker Swarm Mode** (Built-in Orchestration):
  - Swarm Architecture: Managers vs. Workers.
  - Creating a Swarm: `docker swarm init`
  - Deploying a Stack: `docker stack deploy -c docker-compose.yml my-app`
  - Services, Replicas, and Rolling Updates.
- **Kubernetes (The Industry Standard):**
  - High-level concepts: Pods, Deployments, Services.
  - How it compares to Docker Swarm.
  - Using `kubectl` for basic operations.
  - **Note:** This is a vast topic; the goal here is familiarity, not mastery.

Module 12: CI/CD with Docker

- The role of Docker in Continuous Integration and Continuous Deployment.
- Building and pushing Docker images as part of a CI pipeline (e.g., with GitHub Actions, GitLab CI).
- A simple example of a CI pipeline that builds a Docker image, runs tests, and pushes to a registry.

Part 4: Capstone Project & Best Practices

Goal: Solidify all learned concepts by building a real-world application.

Module 13: Capstone Project

- **Task:** Students build a complete, multi-service application (e.g., a blog with a frontend, backend API, and database).
- **Requirements:**

1. Each service has its own Dockerfile.
2. Use a `docker-compose.yml` file to define the entire stack.
3. Use custom Docker networks.
4. Use named volumes for persistent data (DB).
5. (Optional) Configure environment variables for different configurations (dev vs prod).
6. (Optional) Deploy the stack to a cloud VM using Docker Compose or a single-node Swarm.

Module 14: Development & Production Best Practices

- Development: Using `docker-compose.override.yml` for development-specific settings.
- Production: Writing secure and efficient Dockerfiles.
- Logging strategies.
- Healthchecks.
- Monitoring containers

Module 1: Introduction to Containerization

1. The Problem: "It works on my machine!"

The Universal Developer Dilemma:

```
Developer A: "The app runs perfectly on my laptop!"
Developer B: "It crashes on my machine with this weird error..."
Tester: "It's not working in the staging environment!"
Production: "The app is down! Everything was working in dev!"
```

Root Causes:

- Different operating systems (Windows vs. Mac vs. Linux)
- Different software versions (Node.js 14 vs 18, Python 2 vs 3)
- Different dependency packages and libraries
- Different environment variables and configurations
- Different system architectures and hardware

Result: Wasted hours debugging environment-specific issues instead of building features.

2. The Solution: What is Containerization?

Containerization is a standardized way to package an application along with:

- All its dependencies
- Configuration files
- System libraries
- Environment variables
- Runtime (Node.js, Python, Java, etc.)

Think of it like a Shipping Container:

- Standardized size and interface
- Contents are isolated and protected
- Can be moved anywhere (ship, train, truck) without opening
- Works the same way in any port worldwide

Technical Definition:
A container is a lightweight, standalone, executable package of software that includes everything needed to run it: code, runtime, system tools, system libraries, and settings.

3. Docker vs. Virtual Machines: The Evolution

Traditional Virtual Machines (VMs):

```
+-------------------+-------------------+-------------------+
|      App A        |      App B        |      App C        |
|   Binary/Libs     |   Binary/Libs     |   Binary/Libs     |
+-------------------+-------------------+-------------------+
|   Guest OS A      |   Guest OS B      |   Guest OS C      |
|  (Full OS Copy)   |  (Full OS Copy)   |  (Full OS Copy)   |
+-----------------------------------------------------------+
|                Hypervisor (Virtualization Layer)          |
+-----------------------------------------------------------+
|                   Host Operating System                   |
+-----------------------------------------------------------+
|                     Physical Server                      |
+-----------------------------------------------------------+
```

VM Characteristics:

- **Heavyweight:** Each VM includes a FULL operating system (GBs)
- **Slow to start:** Boots entire OS (minutes)
- **High resource overhead:** Duplicate OS kernels in memory
- **Limited density:** Fewer VMs per physical server

Docker Containers:

```
+-------------------+-------------------+-------------------+
|   Container A     |   Container B     |   Container C     |
|      App A        |      App B        |      App C        |
|   Binary/Libs     |   Binary/Libs     |   Binary/Libs     |
+-----------------------------------------------------------+
|                Docker Engine (Container Runtime)          |
+-----------------------------------------------------------+
|                   Host Operating System                   |
+-----------------------------------------------------------+
|                     Physical Server                      |
+-----------------------------------------------------------+
```

Container Characteristics:

- **Lightweight:** Shares host OS kernel (MBs)
- **Fast to start:** Milliseconds to seconds
- **Low overhead:** Minimal resource duplication
- **High density:** Hundreds of containers per server

4. Key Docker Concepts

Docker Image

- **What it is:** A read-only template with instructions for creating a container
- **Analogy:** A class in Object-Oriented Programming (blueprint)
- **Contains:** Application code + dependencies + environment settings
- **Format:** Layered architecture (each instruction in Dockerfile creates a layer)
- **Examples:** `nginx:latest`, `python:3.9`, `node:16-alpine`

Docker Container

- **What it is:** A runnable instance of an image
- **Analogy:** An object instance created from a class
- **Characteristics:**
  - Isolated process running on the host
  - Has its own filesystem, networking, and isolated process tree
  - Ephemeral (can be stopped, started, deleted)
- **Created from:** `docker run image_name`

Docker Registry

- **What it is:** A storage and distribution system for Docker images
- **Public Registry: Docker Hub** (hub.docker.com) - like GitHub for Docker images
- **Private Registries:** AWS ECR, Google Container Registry, Azure Container Registry
- **Function:** Store, version, and distribute images across teams

Relationship:

```
Dockerfile --build--> Image --run--> Container
    ↓
  push/pull
    ↓
Registry (Docker Hub)
```

5. The Docker Ecosystem

Docker Engine

- The core product, the heart of Docker
- **Components:**
  - **Docker Daemon (`dockerd`)**: Background service managing containers
  - **Docker CLI (`docker`)**: Command-line interface to talk to the daemon
  - **REST API**: Interface for programs to talk to the daemon

Docker Desktop

- **For:** Windows and Mac users
- **Includes:** Docker Engine, Docker CLI, Docker Compose, Kubernetes
- **Provides:** Easy GUI and seamless integration with host OS

Docker CLI

- The primary way to interact with Docker
- **Commands:** `docker run`, `docker build`, `docker ps`, etc.
- **Syntax:** `docker [OPTIONS] COMMAND [ARG...]`

Module 2: Getting Started & Installation

1. Installing Docker

Windows:

1. Download Docker Desktop from docker.com
2. Requires Windows 10/11 Pro, Enterprise, or Education
3. For Windows Home: Use WSL 2 backend
4. Installation enables Hyper-V and containers features

Mac:

1. Download Docker Desktop from docker.com
2. Works on both Intel and Apple Silicon Macs
3. Automatically sets up virtualization

Linux (Ubuntu Example):

bash

```bash
# Update package index
sudo apt-get update

# Install prerequisites
sudo apt-get install apt-transport-https ca-certificates curl gnupg lsb-release

# Add Docker's official GPG key
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /u
sr/share/keyrings/docker-archive-keyring.gpg

# Add stable repository
echo "deb [arch=amd64 signed-by=/usr/share/keyrings/docker-archive-keyring.gpg] ht
tps://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/
apt/sources.list.d/docker.list > /dev/null

# Install Docker Engine
sudo apt-get update
sudo apt-get install docker-ce docker-ce-cli containerd.io

# Add user to docker group (to avoid using sudo)
sudo usermod -aG docker $USER
# Log out and log back in for group changes to take effect
```

2. Verifying Installation

```
# Check Docker version
docker --version
# Output: Docker version 20.10.12, build e91ed57

# Check detailed version information
docker version
# Shows Client and Server version details

# Check system information
docker info
# Shows containers running, images, storage driver, etc.
```

3. Running Your First Container

The Hello World Moment:

```
docker run hello-world
```

What happens behind the scenes:

1. Docker CLI contacts Docker Daemon: "Run hello-world"
2. Daemon checks locally: "Do I have the `hello-world` image?"
3. If not found locally, downloads from Docker Hub
4. Creates a new container from the image
5. Executes the container (which prints a welcome message)
6. Container stops (its work is done)

Expected Output:

```
Hello from Docker!
This message shows that your installation appears to be working correctly.
```

4. Introduction to Docker CLI

Basic Command Structure:

```
docker [OPTIONS] COMMAND [ARG...]
```

Essential Commands for Beginners:

Container Management:

```
# Run a container
docker run nginx

# Run in detached mode (background)
docker run -d nginx

# Run with interactive terminal
docker run -it ubuntu bash

# List running containers
docker ps

# List all containers (including stopped)
docker ps -a

# Stop a container
docker stop <container_id>

# Remove a container
docker rm <container_id>
```

Image Management:

```
bash

# List downloaded images
docker images

# Pull an image without running it
docker pull nginx

# Remove an image
docker rmi <image_name>
```

Common Options/Flags:

- `-d` or `--detach`: Run container in background
- `-p` or `--publish`: Map container port to host port `-p 8080:80`
- `-v` or `--volume`: Mount a volume `-v /host/path:/container/path`
- `--name`: Give container a custom name `--name my-container`
- `-it`: Interactive mode with pseudo-TTY (for shells)
- `-e` or `--env`: Set environment variables `-e MY_VAR=value`

5. Understanding the Architecture

```
+-------------------------------------------------+
|                 DOCKER HOST                     |
|                                                 |
|  +-------------+    +-------------+            |
|  | Container A |    | Container B |            |
|  | (nginx)     |    | (redis)     |            |
|  +-------------+    +-------------+            |
|                                                 |
|  +---------------------------------------------+ |
|  |              DOCKER ENGINE                 | |
|  |  +-----------+     +-------------------+   | |
|  |  | Images    |     | Containers        |   | |
|  |  | (Layers)  |     | (Running Apps)    |   | |
|  |  +-----------+     +-------------------+   | |
|  +---------------------------------------------+ |
|                                                 |
|  +---------------------------------------------+ |
|  |               HOST OS                     | |
|  +---------------------------------------------+ |
+-------------------------------------------------+
         ↑
         | docker commands
+-------------------------------------------------+
|                 DOCKER CLI                      |
|   (Terminal where you type commands)           |
+-------------------------------------------------+
```

Hands-On Practice Exercises:

1. **Verify your installation:**

```
docker --version
docker info
```

2. **Run the hello-world container:**

```
docker run hello-world
```

3. **Run an Nginx web server:**

```
docker run -d -p 8080:80 --name my-webserver nginx
```

Then visit `http://localhost:8080` in your browser

4. **Explore running containers:**

```
docker ps
docker logs my-webserver
```

5. **Clean up:**

```
docker stop my-webserver
docker rm my-webserver
```

Module 3: The Dockerfile - Building Custom Images

1. What is a Dockerfile?

A **Dockerfile** is a text document that contains all the commands a user could call on the command line to assemble an image. It's the blueprint or recipe for creating Docker images.

Analogy:

- **Dockerfile** = Recipe (instructions to prepare a dish)
- **Docker Image** = Prepared dish (ready to serve)
- **Docker Container** = Actually eating the dish (running instance)

Key Characteristics:

- Plain text file named `Dockerfile` (no extension)
- Contains sequential instructions/commands
- Each instruction creates a new layer in the image
- Follows specific syntax and conventions

2. Common Dockerfile Instructions

FROM - The Foundation

dockerfile

```
FROM <image>[:<tag>] [AS <name>]
```

- **Purpose:** Sets the base image for subsequent instructions
- **Must be** the first non-comment instruction in a Dockerfile
- **Best Practice:** Use specific tags, not `latest`

Examples:

dockerfile

```
FROM ubuntu:20.04
FROM python:3.9-slim
FROM node:16-alpine
FROM nginx:1.21
```

RUN - Executing Commands During Build

dockerfile

```
RUN <command>                          # Shell form
RUN ["executable", "param1", "param2"] # Exec form
```

- **Purpose:** Executes commands in a new layer on top of the current image
- **Common uses:** Installing packages, creating directories, downloading files

Examples:

dockerfile

```
# Shell form (uses /bin/sh -c)
RUN apt-get update && apt-get install -y python3

# Exec form (direct execution)
RUN ["/bin/bash", "-c", "echo 'Hello World'"]

# Best practice - chain commands to reduce layers
RUN apt-get update \
    && apt-get install -y \
        python3 \
        python3-pip \
    && rm -rf /var/lib/apt/lists/*
```

COPY & ADD - Adding Files

COPY

dockerfile

```
COPY <src> <dest>
```

- **Purpose:** Copies files/directories from the host to the image
- **Simple and predictable**
- **Best Practice:** Use COPY unless you need ADD's extra features

Examples:

dockerfile

```
COPY package.json ./
COPY src/ /app/src/
COPY . /app/
```

ADD

dockerfile

```
ADD <src> <dest>
```

- **Purpose:** COPY with extra features
- **Additional capabilities:**
  - Can copy from URLs
  - Can automatically extract tar archives
- **Best Practice:** Prefer COPY for simplicity unless you need these features

Examples:

```
ADD https://example.com/file.tar.gz /tmp/
ADD app.tar.gz /app/  # Extracts automatically
```

WORKDIR - Setting Working Directory

```
WORKDIR /path/to/directory
```

- **Purpose:** Sets the working directory for any RUN, CMD, ENTRYPOINT, COPY, and ADD instructions
- **Creates directory** if it doesn't exist
- **Similar to** `cd` command
- **Best Practice:** Use absolute paths

Examples:

dockerfile

```
WORKDIR /app
COPY . .    # Copies to /app

WORKDIR /src
RUN npm install  # Runs in /src
```

EXPOSE - Documenting Ports

dockerfile

```
EXPOSE <port> [<port>/<protocol>]
```

- **Purpose:** Documents which ports the container listens on
- **Does NOT** actually publish the port
- **Informational** only - used for documentation and with `docker run -P`

Examples:

```
EXPOSE 80
EXPOSE 3000
EXPOSE 5432/tcp
EXPOSE 80/tcp 443/tcp
```

CMD & ENTRYPOINT - Default Commands

CMD - Default Command

```dockerfile
CMD ["executable","param1","param2"] # Exec form (recommended)
CMD command param1 param2            # Shell form
```

- **Purpose:** Provides defaults for executing the container
- **Can be overridden** when running the container
- **Only one CMD** per Dockerfile (if multiple, last one wins)

Examples:

```dockerfile
CMD ["nginx", "-g", "daemon off;"]
CMD ["python", "app.py"]
CMD echo "Hello World"
```

ENTRYPOINT - Container Main Command

```dockerfile
ENTRYPOINT ["executable", "param1", "param2"] # Exec form
ENTRYPOINT command param1 param2              # Shell form
```

- **Purpose:** Configures a container that will run as an executable
- **Not easily overridden** (use `--entrypoint` flag)
- **CMD becomes arguments** to ENTRYPOINT

Examples:

```
ENTRYPOINT ["docker-entrypoint.sh"]
ENTRYPOINT ["python"]
CMD ["app.py"]  # Becomes: python app.py
```

CMD vs ENTRYPOINT:

Scenario	ENTRYPOINT	CMD	Result
Both defined	`["python"]`	`["app.py"]`	`python app.py`
Only CMD	-	`["python", "app.py"]`	`python app.py`
Only ENTRYPOINT	`["python"]`	-	`python`
Override CMD	`["python"]`	`["app.py"]` + `docker run my-image test.py`	`python test.py`

3. **Other Useful Instructions**

ENV - Environment Variables

```
ENV <key>=<value> ...
dockerfile

ENV NODE_ENV=production
ENV APP_PORT=3000 DB_HOST=localhost
```

ARG - Build-time Variables

```
ARG <name>[=<default value>]
dockerfile

ARG VERSION=latest
FROM python:${VERSION}
```

USER - Set User

```
USER <user>[:<group>]
dockerfile

USER nobody
USER 1000:1000
```

4. Hands-on Lab 1: Create a Simple Web Application

Option A: Python Flask Application

Step 1: Create project structure

```
mkdir python-docker-app && cd python-docker-app
```

Step 2: Create app.py

python

```python
from flask import Flask
app = Flask(__name__)

@app.route('/')
def hello():
    return '<h1>Hello from Docker Python App!</h1><p>This is running in a containe
r.</p>'

@app.route('/health')
def health():
    return {'status': 'healthy'}

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000, debug=True)
```

Step 3: Create requirements.txt

```
Flask==2.3.3
```

Step 4: Create Dockerfile

```
# Use official Python runtime as base image
FROM python:3.9-slim
# Set working directory in container
WORKDIR /app
# Copy requirements first (for better caching)
COPY requirements.txt .
# Install Python dependencies
RUN pip install --no-cache-dir -r requirements.txt
# Copy application code
COPY . .
```

```
# Expose port
EXPOSE 5000
# Define environment variable
ENV FLASK_APP=app.py
# Run the application
CMD ["python", "app.py"]
```

Option B: Node.js Express Application

Step 1: Create project structure

```
mkdir node-docker-app && cd node-docker-app
```

Step 2: Create package.json

```
{
  "name": "node-docker-app",
  "version": "1.0.0",
  "description": "Simple Node.js Docker app",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2"
  }
}
```

Step 3: Create app.js

```
const express = require('express');
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  res.send(`
    <h1>Hello from Docker Node.js App!</h1>
    <p>This is running in a container.</p>
    <p>Server time: ${new Date().toISOString()}</p>
  `);
});

app.get('/health', (req, res) => {
```

```
  res.json({ status: 'healthy', timestamp: new Date().toISOString() });
});

app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 4: Create Dockerfile

```
# Use official Node.js runtime as base image
FROM node:16-alpine
# Set working directory in container
WORKDIR /app
# Copy package files
COPY package*.json ./
# Install dependencies
RUN npm install
# Copy application code
COPY . .
# Expose port
EXPOSE 3000
# Define environment variable
ENV NODE_ENV=production
# Run the application
CMD ["npm", "start"]
```

Step 5: Build the Image

```
# Build the image
docker build -t my-python-app .

# Or for Node.js
docker build -t my-node-app .

# View built images
docker images
```

Module 4: Container Management Basics

1. Core Docker Commands

docker run - The Most Important Command

```
docker run [OPTIONS] IMAGE [COMMAND] [ARG...]
```

Common Flags:

- `-d` or `--detach`: Run container in background
- `-p` or `--publish`: Map container port to host port `host_port:container_port`
- `-v` or `--volume`: Mount volumes `host_path:container_path`
- `--name`: Assign a name to the container
- `-it`: Interactive mode with pseudo-TTY
- `-e` or `--env`: Set environment variables
- `--rm`: Automatically remove container when it exits
- `-P`: Publish all exposed ports to random ports

Examples:

```
# Simple run
docker run nginx

# Run in background
docker run -d nginx

# Run with port mapping
docker run -d -p 8080:80 nginx

# Run with name and volume
docker run -d --name my-app -p 3000:3000 -v $(pwd):/app my-node-app

# Run with environment variables
docker run -d -e NODE_ENV=production -e PORT=3000 my-node-app

# Interactive container
docker run -it ubuntu bash

# Run and auto-remove
docker run --rm -it python:3.9 python
```

docker ps - Listing Containers

```
docker ps [OPTIONS]
```

Common Flags:

- `-a` or `--all`: Show all containers (default shows only running)
- `-q` or `--quiet`: Only display numeric IDs
- `--no-trunc`: Don't truncate output

Examples:

```
# Show running containers
docker ps

# Show all containers (including stopped)
docker ps -a

# Show only container IDs
docker ps -q

# Show with full output
docker ps --no-trunc
```

Output Format:

```
CONTAINER ID   IMAGE     COMMAND                  CREATED         STATUS         PORTS                   NAMES
a1b2c3d4e5f6   nginx     "/docker-entrypoint.…"   2 minutes ago   Up 2 minutes   0.0.0.0:8080->80/tcp    my-nginx
```

docker stop/start/restart - Managing Container State

```
docker stop [OPTIONS] CONTAINER [CONTAINER...]
docker start [OPTIONS] CONTAINER [CONTAINER...]
docker restart [OPTIONS] CONTAINER [CONTAINER...]
```

Examples:

```
# Stop a container
docker stop my-container
docker stop a1b2c3d4e5f6
# Start a stopped container
docker start my-container
# Restart a container
docker restart my-container
# Stop multiple containers
docker stop container1 container2 container3

# Stop all running containers
docker stop $(docker ps -q)
```

docker rm - Removing Containers

```
docker rm [OPTIONS] CONTAINER [CONTAINER...]
```

Common Flags:

- `-f` or `--force`: Force removal of running container
- `-v` or `--volumes`: Remove volumes associated with container

Examples:

```
# Remove stopped container
docker rm my-container
# Force remove running container
docker rm -f my-container
# Remove container and its volumes
docker rm -v my-container
# Remove all stopped containers
docker container prune
# Remove multiple containers
docker rm container1 container2
# Remove all containers (be careful!)
docker rm -f $(docker ps -aq)
```

docker logs - Viewing Container Output

```
docker logs [OPTIONS] CONTAINER
```

Common Flags:

- `-f` or `--follow`: Follow log output
- `--tail N`: Show last N lines
- `-t` or `--timestamps`: Show timestamps
- `--since TIME`: Show logs since timestamp

Examples:

```
# View logs
docker logs my-container
# Follow logs (like tail -f)
docker logs -f my-container
# Show last 10 lines
docker logs --tail 10 my-container
# Show logs with timestamps
docker logs -t my-container
# Show logs from last hour
docker logs --since 1h my-container
```

docker exec - Executing Commands in Running Containers

```
docker exec [OPTIONS] CONTAINER COMMAND [ARG...]
```

Common Flags:

- `-it`: Interactive mode with pseudo-TTY
- `-e`: Set environment variables
- `-w`: Working directory inside the container

Examples:

```
# Run command in container
docker exec my-container ls -la
# Open interactive shell in running container
docker exec -it my-container bash
docker exec -it my-container sh
# Run command with environment variable
docker exec -e MY_VAR=value my-container env
# Run command in specific directory
docker exec -w /app my-container pwd
# Check processes in container
docker exec my-container ps aux
```

2. Hands-on Lab 2: Run and Manage Custom Application

Step 1: Build and Run Your Application

```
# Navigate to your project directory
cd python-docker-app  # or node-docker-app
# Build the image (if not already done)
docker build -t my-web-app .
# Run the container in detached mode with port mapping
docker run -d -p 5000:5000 --name my-running-app my-web-app
# For Node.js app, use port 3000
docker run -d -p 3000:3000 --name my-running-app my-web-app
```

Step 2: Verify Container is Running

```
# Check running containers
docker ps
# Expected output:
# CONTAINER ID   IMAGE        COMMAND          CREATED         STATUS         PORTS                    NAMES
# a1b2c3d4e5f6   my-web-app   "python app.py"  10 seconds ago  Up 9 seconds   0.0.0.0:5000->5000/tcp   my-running-app
```

Step 3: Test Your Application

```
# Open web browser and visit:
# http://localhost:5000  (Python)
# http://localhost:3000  (Node.js)
# Or use curl to test
curl http://localhost:5000
curl http://localhost:5000/health
```

Step 4: View Container Logs

```
# View logs
docker logs my-running-app
# Follow logs in real-time
docker logs -f my-running-app
# View last 5 lines
docker logs --tail 5 my-running-app
```

Step 5: Execute Commands in Container

```
# Open shell inside the container
docker exec -it my-running-app bash
# For Alpine-based images use: sh
# Once inside, explore:
ls -la
pwd
cat app.py  # or cat app.js
ps aux
exit
```

Step 6: Container Management Practice

```
# Stop the container
docker stop my-running-app
# Verify it's stopped
docker ps -a
# Start it again
docker start my-running-app
# Restart it
docker restart my-running-app
# Check resource usage
docker stats my-running-app
```

Step 7: Clean Up

```
# Stop and remove the container
docker stop my-running-app
docker rm my-running-app
# Remove the image (optional)
docker rmi my-web-app
```

3. Common Workflow Summary

```
# 1. Build image from Dockerfile
docker build -t my-app .
# 2. Run container
docker run -d -p 8080:80 --name my-container my-app
# 3. Check status
docker ps
docker logs my-container
# 4. Test application
curl http://localhost:8080
# 5. Debug if needed
docker exec -it my-container bash
# 6. Stop and clean up
docker stop my-container
docker rm my-container
```

4. Quick Reference Cheat Sheet

```
# Container Management
docker run -d -p HOST:CONTAINER --name NAME IMAGE
docker ps
docker ps -a
docker stop NAME
docker start NAME
docker restart NAME
docker rm NAME
docker rm -f NAME
# Image Management
docker build -t NAME .
docker images
docker rmi IMAGE
# Debugging
docker logs NAME
docker logs -f NAME
docker exec -it NAME COMMAND
docker inspect NAME
# Cleanup
docker container prune    # Remove stopped containers
docker image prune        # Remove unused images
docker system prune       # Remove all unused data
```

Module 5: Docker Images Deep Dive

1. Image Layers and the Union File System

What are Image Layers?

Docker images are built using a **layered architecture**. Each instruction in a Dockerfile creates a new layer on top of the previous ones.

Example Dockerfile:

```
FROM ubuntu:20.04          # Layer 1: Base OS layer
RUN apt-get update         # Layer 2: Package list update
RUN apt-get install -y nodejs  # Layer 3: Node.js installation
COPY . /app                # Layer 4: Application code
CMD ["node", "app.js"]     # Layer 5: Metadata layer
```

Union File System (UnionFS)

- **Concept:** Multiple layers are stacked and presented as a single unified filesystem
- **Read-only layers:** All layers except the topmost are read-only
- **Copy-on-Write (CoW):** When a file is modified, it's copied to the top writable layer

Visual Representation:

```
+----------------------+
|   Container Layer    | ← Read-write layer (container-specific)
|     (Writable)       |
+----------------------+
|   COPY . /app        | ← Layer 4 (Read-only)
+----------------------+
|   RUN apt-get install| ← Layer 3 (Read-only)
+----------------------+
|   RUN apt-get update | ← Layer 2 (Read-only)
+----------------------+
|   FROM ubuntu:20.04  | ← Layer 1 (Read-only)
+----------------------+
```

Benefits of Layered Architecture:

1. **Storage Efficiency:** Layers are shared between images
2. **Build Caching:** Unchanged layers are reused
3. **Faster Downloads:** Only new layers are downloaded
4. **Smaller Updates:** Only changed layers need updating

Inspecting Image Layers:

```
# Show image history and layers

docker image history my-app
# Detailed image inspection
docker image inspect my-app
# Show layer sizes
docker system df
```

Example Output:

```
IMAGE          CREATED        CREATED BY                                      SIZE
a1b2c3d4e5f6   2 minutes ago  CMD ["node" "app.js"]                           0B
f1e2d3c4b5a6   2 minutes ago  COPY . /app # buildkit                          15.2MB
c1b2a3d4e5f6   2 minutes ago  RUN /bin/sh -c apt-get install -y nodejs # …    152.1MB
b1a2c3d4e5f6   3 minutes ago  RUN /bin/sh -c apt-get update # buildkit        21.5MB
e1d2c3b4a5f6   4 minutes ago  /bin/sh -c #(nop)  CMD ["/bin/bash"]            0B
```

2. Efficient Dockerfiles: Leveraging Layer Caching

Understanding Build Cache

Docker caches each layer during build. When rebuilding, it checks:

1. **Instruction match:** Is the instruction the same as previous build?
2. **Parent layer match:** Is the parent layer the same?
3. **Build context changes:** Have files changed for COPY/ADD?

Optimization Strategies:

1. Order Instructions from Least to Most Frequently Changing

```
# ❌ INEFFICIENT - Code changes break cache early
FROM node:16
COPY . /app          # Changes frequently - cache breaks here
RUN npm install      # Always runs even if only code changes
CMD ["npm", "start"]

# ✅ EFFICIENT - Dependencies cached separately
FROM node:16
COPY package*.json ./   # Infrequently changes
RUN npm install         # Cached if package.json unchanged
COPY . /app            # Changes frequently - but last
CMD ["npm", "start"]
```

2. Combine RUN Commands

```
# ❌ INEFFICIENT - Multiple layers
RUN apt-get update
RUN apt-get install -y python3
RUN apt-get install -y python3-pip
RUN rm -rf /var/lib/apt/lists/*

# ✅ EFFICIENT - Single layer, smaller image
RUN apt-get update \
    && apt-get install -y \
        python3 \
        python3-pip \
    && rm -rf /var/lib/apt/lists/*
```

3. Use Specific Base Images

```
# ❌ HEAVY - Full OS
FROM ubuntu:20.04

# ✅ LIGHTWEIGHT - Minimal version
FROM python:3.9-slim
FROM node:16-alpine
```

.dockerignore File

Prevents unnecessary files from being sent to Docker daemon, reducing build context size and preventing cache invalidation.

.dockerignore Example:

```
# Ignore everything by default
*
# Allow specific files
!package.json
!package-lock.json
!src/
!public/
# Exclude common development files
.git
.gitignore
README.md
Dockerfile
.dockerignore
node_modules
npm-debug.log
.env
.nyc_output
coverage/
.DS_Store
```

3. Multi-stage Builds

The Problem:

Development tools, compilers, and build dependencies increase image size but aren't needed at runtime.

The Solution: Multi-stage Builds

Use multiple FROM statements to separate build environment from runtime environment.

Example: Node.js Application

```
# Stage 1: Build stage
FROM node:16 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
# Stage 2: Runtime stage
FROM node:16-alpine
WORKDIR /app
COPY --from=builder /app/node_modules ./node_modules
COPY . .
USER node
EXPOSE 3000
CMD ["node", "server.js"]
```

Benefits of Multi-stage Builds:

- **Smaller final images** (no build tools included)
- **Better security** (fewer packages = smaller attack surface)
- **Clean separation** of build and runtime concerns

Size Comparison:

```
# Single stage build
docker images
# REPOSITORY  TAG     IMAGE ID      SIZE
# my-app      old      a1b2c3d4e5f6  1.2GB
# Multi-stage build
docker images
# REPOSITORY  TAG     IMAGE ID      SIZE
# my-app      new      b2c3d4e5f6a1  180MB
```

4. Image Tagging Best Practices

Tagging Syntax:

```
docker tag SOURCE_IMAGE[:TAG] TARGET_IMAGE[:TAG]
```

Best Practices:

1. Use Semantic Versioning

```
# ✅ GOOD - Specific versions
docker build -t my-app:1.0.0 .
docker build -t my-app:1.0.1 .
docker build -t my-app:2.0.0 .
# ❌ BAD - Only using latest
docker build -t my-app:latest .
```

2. Include Environment and Metadata

```
# Environment specific
docker build -t my-app:1.0.0-dev .
docker build -t my-app:1.0.0-staging .
docker build -t my-app:1.0.0-prod .
# Git commit based
docker build -t my-app:$(git rev-parse --short HEAD) .
```

3. Registry Namespacing

```
# For Docker Hub
docker build -t username/my-app:1.0.0 .
# For private registries
docker build -t myregistry.com/team/my-app:1.0.0 .
docker build -t 123456789012.dkr.ecr.us-east-1.amazonaws.com/my-app:1.0.0 .
```

4. Multi-architecture Tags

```
# For different CPU architectures
docker build -t my-app:1.0.0-amd64 .
docker build -t my-app:1.0.0-arm64 .
```

5. Image Management Commands

docker build

```
# Basic build
docker build -t my-app .
# Build with specific Dockerfile
docker build -f Dockerfile.prod -t my-app:prod .
# Build without cache
docker build --no-cache -t my-app .
# Build with build arguments
docker build --build-arg NODE_ENV=production -t my-app .
# Multi-stage build target
docker build --target builder -t my-app:builder .
```

docker images

```
# List images
docker images
# List with custom format
docker images --format "table {{.Repository}}\t{{.Tag}}\t{{.Size}}"
# Filter images
docker images --filter "dangling=true"
docker images --filter "label=version=1.0"
docker images my-app*
# Show only image IDs
docker images -q
```

docker rmi (Remove Image)

```
# Remove specific image
docker rmi my-app:1.0.0
# Remove by ID
docker rmi a1b2c3d4e5f6
# Force remove (even if used by containers)
docker rmi -f my-app:1.0.0
# Remove multiple images
docker rmi my-app:1.0.0 my-app:1.0.1
# Remove all unused images
docker image prune
# Remove dangling images (untagged)
docker image prune -a
```

docker tag

```
# Tag an image
docker tag my-app:1.0.0 my-app:latest
docker tag my-app:1.0.0 my-registry.com/my-app:1.0.0

# Tag from one repository to another
docker tag old-repo:tag new-repo:tag
```

Additional Useful Commands:

```
# Save image to tar file
docker save -o my-app.tar my-app:1.0.0
# Load image from tar file
docker load -i my-app.tar
# Push to registry
docker push my-app:1.0.0
# Pull from registry
docker pull my-app:1.0.0
# Inspect image details
docker image inspect my-app:1.0.0
# Show image history
docker history my-app:1.0.0
```

6. Hands-on Lab: Image Optimization

Exercise: Optimize a Dockerfile

Initial Inefficient Dockerfile:

```
FROM ubuntu:20.04
COPY . /app
RUN apt-get update
RUN apt-get install -y nodejs npm python3
RUN cd /app && npm install
RUN chmod -R 777 /app
EXPOSE 3000
CMD ["node", "/app/server.js"]
```

Optimized Dockerfile:

```
# Multi-stage build for Node.js app
FROM node:16 AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci --only=production
FROM node:16-slim
WORKDIR /app
# Create non-root user for security
RUN groupadd -r appuser && useradd -r -g appuser appuser
COPY --from=builder /app/node_modules ./node_modules
COPY . .
# Set proper permissions
RUN chown -R appuser:appuser /app
USER appuser
EXPOSE 3000
CMD ["node", "server.js"]
```

Build and Compare:

```
# Build original
docker build -f Dockerfile.original -t my-app:original .
# Build optimized
docker build -f Dockerfile.optimized -t my-app:optimized .
# Compare sizes
docker images my-app
# Expected output:
# REPOSITORY  TAG        SIZE
# my-app      original   1.1GB
# my-app      optimized  230MB
```

Image Analysis:

```
# Analyze layer sizes
docker history my-app:original
docker history my-app:optimized
# Check security vulnerabilities
docker scan my-app:optimized
# Test both work the same
docker run -d -p 3000:3000 --name app-original my-app:original
docker run -d -p 3001:3000 --name app-optimized my-app:optimized
```

Module 6: Data Persistence: Volumes & Bind Mounts

1. The Problem: Ephemeral Containers

Container Filesystem Characteristics:

- **Ephemeral:** Data is lost when container stops/removed
- **Isolated:** Each container has its own filesystem
- **Copy-on-Write:** Changes are in container-specific layer
- **Non-persistent:** No data persistence by default

Real-world Scenarios Requiring Persistence:

- **Database files** (PostgreSQL, MySQL, MongoDB)
- **Application logs**
- **Uploaded files** (user images, documents)
- **Configuration data**
- **Cache files**

The Challenge:

```
# Start a PostgreSQL container
docker run -d --name postgres-db postgres:13
# Create some data
docker exec postgres-db psql -U postgres -c "CREATE DATABASE myapp;"
# Stop and remove container
docker stop postgres-db
docker rm postgres-db
# Start new container - data is GONE!
docker run -d --name new-postgres postgres:13
docker exec new-postgres psql -U postgres -c "\l"  # No myapp database
```

2. Bind Mounts: Development Workflow

What are Bind Mounts?

- **Direct mapping** between host directory and container directory
- **Two-way sync:** Changes on host reflect in container and vice versa
- **Perfect for development:** Live code reload, debugging

Syntax:

```
docker run -v /host/path:/container/path image_name
# or
docker run --mount type=bind,source=/host/path,target=/container/path image_name
```

Use Cases:

1. **Development:** Mount source code for live reload
2. **Configuration:** Mount config files
3. **Build artifacts:** Share build outputs between containers

Examples:

```
# Development with live reload
docker run -d \
  -p 3000:3000 \
  -v $(pwd):/app \
  -v /app/node_modules \  # Anonymous volume for node_modules
  my-node-app

# Mount specific configuration file
docker run -d \
  -v $(pwd)/nginx.conf:/etc/nginx/nginx.conf:ro \  # Read-only
  nginx

# Multiple mount points
docker run -d \
  -v $(pwd)/src:/app/src \
  -v $(pwd)/config:/app/config \
  -v $(pwd)/logs:/app/logs \
  my-app
```

Bind Mount Considerations:

- **Performance:** Direct filesystem access (good performance)
- **Portability:** Host path must exist and be accessible
- **Security:** Container can modify host filesystem
- **OS Compatibility:** Path format differs (Windows vs Linux)

3. Volumes: Production Storage

What are Docker Volumes?

- **Managed storage** created and managed by Docker
- **Decoupled** from host filesystem structure
- **Persistent** beyond container lifecycle
- **Portable** across different hosts

Volume Types:

1. **Named Volumes:** Explicitly created and named
2. **Anonymous Volumes:** Auto-created, random names
3. **Host Volumes:** Bind mounts (discussed above)

Volume Management Commands:

docker volume create

```
# Create a named volume
docker volume create myapp-data
# Create with specific driver
docker volume create --driver local \
  --opt type=none \
  --opt device=/home/user/data \
  --opt o=bind \
  myapp-data
# Create with labels
docker volume create --label app=myapp --label env=production db-data
```

docker volume ls

```
# List all volumes
docker volume ls
# Filter volumes
docker volume ls --filter "name=myapp"
docker volume ls --filter "label=env=production"
```

```
# Format output
docker volume ls --format "table {{.Name}}\t{{.Driver}}\t{{.Labels}}"
```

docker volume inspect

```
# Inspect volume details
docker volume inspect myapp-data
# Multiple volumes
docker volume inspect vol1 vol2 vol3
# Specific format
docker volume inspect -f '{{.Mountpoint}}' myapp-data
```

Additional Volume Commands:

```
# Remove volume
docker volume rm myapp-data
# Remove unused volumes
docker volume prune
# Remove all unused volumes (force)
docker volume prune -f
```

Using Volumes with Containers:

```
# Create and use volume
docker run -d \
  --name postgres-db \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:13
# Use existing volume
docker run -d \
  --name mysql-db \
  -v mysql-data:/var/lib/mysql \
  mysql:8.0
# Multiple volumes
docker run -d \
  --name my-app \
  -v app-data:/app/data \
  -v app-logs:/app/logs \
  -v app-cache:/app/cache \
  my-app:latest
```

Volume Drivers:

Docker supports different volume drivers for various storage backends:

- **local:** Default, local host storage
- **nfs:** Network File System
- **sshfs:** SSH File System
- **cloud:** AWS EBS, Azure Disk, Google Persistent Disk

```
# Create NFS volume
docker volume create --driver local \
  --opt type=nfs \
  --opt o=addr=192.168.1.100,rw \
  --opt device=:/path/to/nfs/share \
  nfs-volume
```

4. Bind Mounts vs Volumes: When to Use What

Aspect	Bind Mounts	Volumes
Use Case	Development, config files	Production, databases
Performance	Direct FS access (fast)	Docker-managed (good)
Portability	Host path dependent	Docker-managed paths
Backup	Manual	`docker volume backup` tools
Sharing	Between host and container	Between containers
Persistence	Yes	Yes
Orchestration	Limited	Excellent (Docker Compose, Swarm)

Decision Guide:

- **Development:** Use **bind mounts** for source code
- **Production:** Use **volumes** for application data
- **Configuration:** Use **bind mounts** (read-only) for config files
- **Database:** Always use **volumes** for database files
- **Temporary data:** Use **tmpfs** mounts for non-persistent data

5. Hands-on Lab 3: Data Persistence Practice

Part A: Development with Bind Mounts (Live Reload)

Step 1: Create a Node.js development app

```
mkdir live-reload-app && cd live-reload-app
```

Step 2: Create package.json

```
{
  "name": "live-reload-app",
  "version": "1.0.0",
  "description": "Development app with live reload",
  "main": "app.js",
  "scripts": {
    "start": "node app.js",
    "dev": "nodemon app.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "nodemon": "^3.0.1"
  }
}
```

Step 3: Create app.js

```
const express = require('express');
```

```
const app = express();
const PORT = process.env.PORT || 3000;

app.get('/', (req, res) => {
  const message = process.env.MESSAGE || 'Hello from Docker!';
  res.send(`
    <h1>${message}</h1>
    <p>Server time: ${new Date().toISOString()}</p>
    <p>This is live reload demonstration</p>
  `);
});

app.get('/health', (req, res) => {
  res.json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    environment: process.env.NODE_ENV || 'development'
  });
});

app.listen(PORT, '0.0.0.0', () => {
  console.log(`Server running on port ${PORT}`);
});
```

Step 4: Run with bind mount for development

```
# Run with bind mount for live reload
docker run -d \
  --name dev-app \
  -p 3000:3000 \
  -v $(pwd):/app \
  -w /app \
  -e NODE_ENV=development \
  node:16 \
  npx nodemon app.js

# Check logs to see live reload in action
docker logs -f dev-app
```

Step 5: Test live reload

bash

```bash
# Make a change to app.js - change the message
# Save the file and watch the logs
# The application should restart automatically

# Test the application
curl http://localhost:3000
```

Part B: Database with Volume (Production-like)

Step 1: Create PostgreSQL with volume

```bash
# Create volume for database
docker volume create postgres-data
# Run PostgreSQL with volume
docker run -d \
  --name postgres-db \
  -e POSTGRES_DB=myapp \
  -e POSTGRES_USER=appuser \
  -e POSTGRES_PASSWORD=secret \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:13
# Verify volume was created and used
docker volume ls
docker volume inspect postgres-data
```

Step 2: Create some data

```bash
# Connect to PostgreSQL and create data
docker exec -it postgres-db psql -U appuser -d myapp
# In PostgreSQL shell:
CREATE TABLE users (id SERIAL PRIMARY KEY, name VARCHAR(100), email VARCHAR(100));
INSERT INTO users (name, email) VALUES ('John Doe', 'john@example.com');
INSERT INTO users (name, email) VALUES ('Jane Smith', 'jane@example.com');
SELECT * FROM users;
\q
```

Step 3: Test persistence

```
# Stop and remove the container (data should persist in volume)
docker stop postgres-db
docker rm postgres-db

# Start a new PostgreSQL container with the same volume
docker run -d \
  --name new-postgres \
  -e POSTGRES_DB=myapp \
  -e POSTGRES_USER=appuser \
  -e POSTGRES_PASSWORD=secret \
  -v postgres-data:/var/lib/postgresql/data \
  -p 5432:5432 \
  postgres:13

# Verify data persists
docker exec -it new-postgres psql -U appuser -d myapp -c "SELECT * FROM users;"
```

Part C: Advanced Volume Operations

Step 1: Backup volume data

```
# Create backup of postgres-data volume
docker run --rm \
  -v postgres-data:/source \
  -v $(pwd)/backups:/backup \
  alpine \
  tar czf /backup/postgres-backup-$(date +%Y%m%d).tar.gz -C /source ./

# Verify backup
ls -la backups/
```

Step 2: Restore volume from backup

```
# Create new volume
docker volume create postgres-restored

# Restore data to new volume
docker run --rm \
  -v postgres-restored:/target \
  -v $(pwd)/backups:/backup \
```

```
  alpine \
  tar xzf /backup/postgres-backup-$(date +%Y%m%d).tar.gz -C /target

# Test restored volume
docker run -d \
  --name test-restored \
  -v postgres-restored:/var/lib/postgresql/data \
  -e POSTGRES_PASSWORD=test \
  postgres:13

docker exec test-restored psql -U postgres -c "\l"
```

Cleanup:

```
# Stop and remove all containers
docker stop dev-app new-postgres test-restored
docker rm dev-app new-postgres test-restored
# Remove volumes (optional)
docker volume rm postgres-data postgres-restored
# Remove unused volumes
docker volume prune
```

Module 7: Networking in Docker

1. How Containers Communicate

The Networking Challenge:

- Containers are isolated by default
- Each container has its own network namespace
- Need controlled communication between containers and external world

Docker Network Architecture:

```
+-----------------------------------------------------+
|                   DOCKER HOST                       |
|                                                     |
|  +-------------+    +-------------+                |
|  | Container A |    | Container B |                |
|  | 172.17.0.2  |    | 172.17.0.3  |                |
|  +-------------+    +-------------+                |
|          |                |                        |
|  +-----------------------------------------------+
|  |            docker0 (Bridge)                  | 172.17.0.1
|  +-----------------------------------------------+
|                              | eth0
|  +-----------------------------------------------+
|  |            Host Network Stack                | 192.168.1.100
|  +-----------------------------------------------+
+-----------------------------------------------------+
```

2. Default Docker Networks

Docker creates three networks automatically:

bridge Network (Default)

- **Default network** for containers
- **NAT-based** connectivity to external world
- **Internal DNS** for container name resolution
- **IP range:** 172.17.0.0/16

```
# Container uses bridge network by default
docker run -d --name web nginx

# Inspect network
docker network inspect bridge
```

host Network

- **Shares host's network namespace**
- **No network isolation** from host
- **Direct access** to host's network interfaces
- **Better performance** (no NAT)

```
# Use host network
docker run -d --name web --network host nginx

# Access directly on host port 80
curl http://localhost
```

none Network

- **Complete network isolation**
- **No network interfaces** (except loopback)
- **Use cases:** Security-sensitive applications, batch processing

```
# No network access
docker run -d --name isolated --network none nginx
# Verify no network
docker exec isolated ip addr show
```

3. Custom Bridge Networks

Why Create Custom Networks?

1. **Isolation:** Separate application stacks
2. **DNS Resolution:** Automatic service discovery
3. **Security:** Network segmentation
4. **Custom Configuration:** Specific IP ranges, gateways

Creating Custom Networks:

```
# Create a custom bridge network
docker network create my-app-network
# Create with specific subnet
docker network create \
  --driver bridge \
  --subnet=10.10.0.0/16 \
  --gateway=10.10.0.1 \
  my-custom-network
# Create with labels
docker network create \
  --label environment=production \
  --label project=myapp \
  prod-network
```

Network Drivers:

- **bridge:** Default for standalone containers
- **host:** Remove network isolation
- **none:** No networking
- **overlay:** For Swarm clusters (multi-host)
- **macvlan:** Assign MAC addresses to containers

4. Docker's Embedded DNS

Automatic Service Discovery:

- Each container has a **hostname** (container name)
- **Built-in DNS server** at 127.0.0.11
- **Container name resolution** within same network
- **Automatic updates** when containers start/stop

DNS Resolution Examples:

```
# Create network and containers
docker network create app-net

docker run -d --name web --network app-net nginx
docker run -d --name api --network app-net my-api

# Containers can communicate using names
# From 'api' container: ping web → resolves to web's IP
# From 'web' container: ping api → resolves to api's IP
```

Testing DNS Resolution:

```
# Start a temporary container on the network
docker run -it --rm --network app-net alpine

# Inside container, test DNS
ping web
ping api
nslookup web
cat /etc/resolv.conf
```

5. Network Management Commands

docker network create

```
# Basic network
docker network create my-network

# Advanced options
docker network create \
  --driver bridge \
  --subnet=192.168.100.0/24 \
  --gateway=192.168.100.1 \
  --opt com.docker.network.driver.mtu=1500 \
  my-custom-network

# IPAM (IP Address Management)
docker network create \
```

```
  --subnet=172.20.0.0/16 \
  --ip-range=172.20.10.0/24 \
  --gateway=172.20.10.1 \
  my-ipam-network
```

docker network ls

```
# List all networks
docker network ls

# Filter networks
docker network ls --filter "driver=bridge"
docker network ls --filter "label=environment=production"

# Custom format
docker network ls --format "table {{.Name}}\t{{.Driver}}\t{{.Scope}}"
```

docker network inspect

```
# Inspect network details
docker network inspect bridge

# Specific information
docker network inspect -f '{{.IPAM.Config}}' my-network

# Multiple networks
docker network inspect network1 network2
```

docker network connect/disconnect

```
# Connect container to network
docker network connect my-network existing-container

# Disconnect container from network
docker network disconnect my-network existing-container

# Connect with specific IP
docker network connect --ip 192.168.100.10 my-network container-name
```

Additional Network Commands:

```
# Remove network
docker network rm my-network

# Remove unused networks
docker network prune

# Show container's networks
docker inspect container-name | grep NetworkMode
```

6. Hands-on Lab 4: Multi-container Application

Scenario: Web Application + Database

Step 1: Create custom network

```
# Create isolated network for our application
docker network create --driver bridge app-network

# Verify network creation
docker network ls
docker network inspect app-network
```

Step 2: Create PostgreSQL database container

```
# Create volume for database persistence
docker volume create postgres-data

# Run PostgreSQL on custom network
docker run -d \
  --name postgres-db \
  --network app-network \
  -e POSTGRES_DB=myapp \
  -e POSTGRES_USER=appuser \
  -e POSTGRES_PASSWORD=secretpassword \
  -v postgres-data:/var/lib/postgresql/data \
  postgres:13

# Verify database is running
docker logs postgres-db
```

Step 3: Create web application

Create a simple Node.js app that connects to the database:

app.js:

```
const express = require('express');
const { Client } = require('pg');
const app = express();
const PORT = 3000;
// Database configuration - using container name as host
const dbConfig = {
  host: 'postgres-db',  // Docker DNS resolves this to container IP
  port: 5432,
  database: 'myapp',
  user: 'appuser',
  password: 'secretpassword'
};
const client = new Client(dbConfig);
// Connect to database
client.connect()
  .then(() => console.log('Connected to PostgreSQL'))
  .catch(err => console.error('Connection error', err));
app.get('/', async (req, res) => {
  try {
    const result = await client.query('SELECT NOW() as current_time');
    res.json({
      message: 'Hello from Docker Network Lab!',
      database_time: result.rows[0].current_time,
      container_id: process.env.HOSTNAME || 'unknown'
    });
  } catch (err) {
    res.status(500).json({ error: err.message });
  }
});
app.get('/health', (req, res) => {
  res.json({
    status: 'healthy',
    timestamp: new Date().toISOString(),
    service: 'web-app'
  });
});
app.listen(PORT, '0.0.0.0', () => {
  console.log(`Web app listening on port ${PORT}`);});
```

package.json:

```
{
  "name": "network-lab-app",
  "version": "1.0.0",
  "description": "Docker networking lab application",
  "main": "app.js",
  "scripts": {
    "start": "node app.js"
  },
  "dependencies": {
    "express": "^4.18.2",
    "pg": "^8.11.0"
  }
}
```

Dockerfile:

```
FROM node:16-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["npm", "start"]
```

Step 4: Build and run web application

```
# Build the web application image
docker build -t network-lab-app .
# Run web app on the same custom network
docker run -d \
  --name web-app \
  --network app-network \
  -p 3000:3000 \
  network-lab-app

# Verify both containers are on the same network
docker network inspect app-network
```

Step 5: Test the application

```
# Test web application
curl http://localhost:3000

# Expected response:
# {
#   "message": "Hello from Docker Network Lab!",
#   "database_time": "2023-10-01T12:00:00.123Z",
#   "container_id": "a1b2c3d4e5f6"
# }

# Test health endpoint
curl http://localhost:3000/health
```

Step 6: Verify network communication

```
# Check if web-app can reach postgres-db by name
docker exec -it web-app ping postgres-db

# Check DNS resolution from web-app
docker exec -it web-app nslookup postgres-db

# Check connection to database from web-app
docker exec -it web-app node -e "
const { Client } = require('pg');
const client = new Client({
  host: 'postgres-db',
  user: 'appuser',
  password: 'secretpassword',
  database: 'myapp'
});
client.connect().then(() => {
  console.log('Database connection successful');
  client.end();
}).catch(err => console.error('Connection failed:', err));
"
```

Step 7: Test isolation

```
# Create a container on default bridge network
docker run -d --name isolated-app nginx
# Try to connect to postgres-db from isolated container
docker exec -it isolated-app nslookup postgres-db
# This should FAIL - different networks
# Connect isolated container to app-network
docker network connect app-network isolated-app
# Now try again - should work
docker exec -it isolated-app nslookup postgres-db
```

Advanced Network Scenarios:

Scenario 1: Multiple networks

```
# Create frontend and backend networks
docker network create frontend-net
docker network create backend-net
# Web app on both networks
docker run -d --name web --network frontend-net nginx
docker network connect backend-net web
# API on backend network only
docker run -d --name api --network backend-net my-api
# Database on backend network only
docker run -d --name db --network backend-net postgres
```

Scenario 2: Network aliases

```
# Create container with network aliases
docker run -d \
  --name my-service \
  --network app-network \
  --network-alias api \
  --network-alias backend \
  my-app
```

```
# Other containers can reach it using api or backend
```

Cleanup:

```
# Stop and remove containers
docker stop web-app postgres-db isolated-app
docker rm web-app postgres-db isolated-app
# Remove network
docker network rm app-network frontend-net backend-net
# Remove volume
docker volume rm postgres-data
# Remove image
docker rmi network-lab-app
```

Module 8: Docker Compose: The Multi-Container Orchestrator

1. What is Docker Compose?

The Problem with Multiple Containers:

Managing multi-container applications with individual `docker run` commands becomes complex:

```
# Example: Web app + Database + Cache
docker run -d --name database -v db-data:/data -e POSTGRES_PASSWORD=secret postgre
s:13
docker run -d --name redis -v redis-data:/data redis:6-alpine
docker run -d --name webapp -p 80:3000 -e DB_URL=postgres://... -e REDIS_URL=redis
://... my-webapp
docker run -d --name worker -e DB_URL=postgres://... -e REDIS_URL=redis://... my-w
orker
```

The Solution: Docker Compose

- **Tool for defining and running multi-container Docker applications**
- **Uses YAML file** (`docker-compose.yml`) to configure services
- **Single command** to start/stop entire application stack
- **Service dependencies** and startup order management
- **Integrated networking** and volume management

Docker Compose Workflow:

1. Define app environment with `Dockerfile`
2. Define services in `docker-compose.yml`
3. Run `docker-compose up`

4. Compose starts and runs entire app

2. docker-compose.yml Structure

Basic Structure:

```yaml
version: '3.8'  # Compose file version

services:       # Container definitions
  web:
    build: .
    ports:
      - "5000:5000"

  database:
    image: postgres:13
    environment:
      POSTGRES_DB: myapp

volumes:        # Persistent storage
  db-data:

networks:       # Network configuration
  app-network:
    driver: bridge
```

Compose File Versions:

- **Version 2:** Legacy format
- **Version 3:** Current standard
- **Version 3.8:** Latest features

4. Essential Docker Compose Commands

docker-compose up

```
# Start all services
docker-compose up
# Start in detached mode
docker-compose up -d
# Start specific services
docker-compose up web database
# Force rebuild of images
docker-compose up --build
# Recreate containers even if unchanged
docker-compose up --force-recreate
```

docker-compose down

```
# Stop and remove containers, networks
docker-compose down
# Remove volumes as well
docker-compose down -v
# Remove images
docker-compose down --rmi all
# Remove orphan containers
docker-compose down --remove-orphans
```

docker-compose ps

```
# List running services
docker-compose ps
# Show all services (including stopped)
docker-compose ps -a
# Show service ports
docker-compose ps --services
```

docker-compose logs

```
# Show all service logs
docker-compose logs
# Follow logs
docker-compose logs -f
# Specific service logs
docker-compose logs web
# Show timestamps
docker-compose logs -t
# Tail last N lines
docker-compose logs --tail=100
```

Other Important Commands:

```
# Build images
docker-compose build
# Pull images
docker-compose pull
# Execute command in running service
docker-compose exec web bash
# Run one-off command
docker-compose run --rm web npm test
# Scale services
docker-compose up --scale worker=3
# View service configuration
docker-compose config
# Stop services without removing
docker-compose stop
# Start stopped services
docker-compose start
# Restart services
docker-compose restart
```

Summary of Part 2

In this intermediate section, you've learned:

1. **Image Optimization:** Layered architecture, efficient Dockerfiles, multi-stage builds
2. **Data Management:** Volumes for production, bind mounts for development
3. **Networking:** Custom networks, DNS resolution, service discovery

4. **Orchestration:** Docker Compose for multi-container applications